INF221 — Lecture 4

Today's topics

1. Recap: Big O & Co
2. Solving recurrence equations
3. Probabilistic analysis and randomized algorithms
   A. The technology problem
   B. Indicator random variables
   C. Randomized algorithms
   D. Generating random numbers

Note: See also *Some useful mathematical material* at end of Lecture 3 notes.

Recap: Big O & Co

Asymptotic upper bound: O-notation

$$O(g(n)) = \{\, f(n) |\, \exists c > 0, n_0 > 0 \ni 0 \leq f(n) \leq cg(n)\ \forall n \geq n_0 \} \tag{1}$$

- All $f(n)$ in $O(g(n))$ are asymptotically non-negative
- All $f(n)$ in $O(g(n))$ are asymptotically bounded *from above* by $cg(n)$
- "A function is in $O(g(n))$ if it does not grow faster than $g(n)$ for large enough n"

Examples:

- $n \in O(n)$
- $n \in O(n^2)$
- $5n^2 + 2n - 10 \in O(n^2)$
- $10^{-6}n^3 \notin O(n^2)$

Asymptotic lower bound: Ω-notation

$$\Omega(g(n)) = \{\, f(n) |\, \exists c > 0, n_0 > 0 \ni 0 \leq cg(n) \leq f(n)\ \forall n \geq n_0 \} \tag{2}$$

- All $f(n)$ in $\Omega(g(n))$ are asymptotically non-negative
- All $f(n)$ in $\Omega(g(n))$ are asymptotically bounded *from below* by $cg(n)$
- "A function is in $\Omega(g(n))$ if it grows at least as fast as $g(n)$ for large enough n"

Examples:

- $n \in \Omega(n)$
- $n^2 \in \Omega(n)$
- $5n^2 + 2n - 10 \in \Omega(n^2)$
- $10^6 n^2 \notin \Omega(n^3)$

Asymptotic tight bound: Θ-notation

$$\Theta(g(n)) = \{ f(n) | \exists c_1 > 0, c_2 > 0, n_0 > 0 \ni 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \ \forall n \geq n$$

- All $f(n)$ in $\Theta(g(n))$ are asymptotically non-negative
- All $f(n)$ in $\Theta(g(n))$ are asymptotically bounded *from above and below* by $c_1 g(n)$ and $c_2 g(n)$
- "A function is in $\Theta(g(n))$ if it grows as fast as $g(n)$ for large enough n

Examples:

- $n \in \Theta(n)$
- $5n^2 + 2n - 10 \in \Theta(n^2)$
- $5n^2 + 2n - 10 \notin \Theta(n)$ (grows too fast, upper bound violated)
- $5n^2 + 2n - 10 \notin \Theta(n^3)$ (grows too slowly, lower bound violated)

Proving dominance relations between functions

We say that $g(n)$ *dominates* $f(n)$ if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$. In this case, we have $f(n) = O(g(n))$. "Dominates" roughly means "grows faster than" for large n.

To prove that $g(n)$ dominating $f(n)$ implies $f(n) = O(g(n))$, we start from the defining inequality

$$f(n) \leq cg(n) \iff \frac{f(n)}{g(n)} \leq c \ .$$

Now from the definition of the limes we have

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \implies \forall \epsilon > 0 \ \exists n_0 \in \mathbb{N} \ni \frac{f(n)}{g(n)} \leq \epsilon \ \forall n > n_0 \ ,$$

which is just what we need to fulfill the definition of $O(g(n))$.

Note that dominance implies $O()$, but *not the other way around*: $f(n) = O(g(n))$ *does not imply* $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.

To see this, remember that every function is "in its own big O", i.e., $f(n) = O(f(n))$.

To test for dominance, we simply take the limit, using L'Hôpital's rule if necessary.

Example 1

Let $f(n) = n$ and $g(n) = n^2$. In this case we have

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{n}{n^2} = \lim_{n\to\infty} \frac{1}{n} = 0$$

so n^2 dominates n and $n = O(n^2)$.

Example 2

Let $f(n) = n^3$ and $g(n) = n^2$. In this case we have

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{n^3}{n^2} = \lim_{n\to\infty} n = \infty$$

so n^2 *does not* dominate n^3. But we can easily see that we can swap roles here and following the same steps as in example 1 show that n^3 dominates n^2, i.e., $n^2 = O(n^3)$. Given the transpose symmetry between O and Ω we also have $n^3 = \Omega(n^2)$. Indeed, if $\lim_{n\to\infty} f(n)/g(n) = \infty$, we can conclude directly that $f(n) = \Omega(g(n))$.

Example 3

Let $f(n) = 2n^2$ and $g(n) = n^2$. In this case we have

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{2n^2}{n^2} = \lim_{n\to\infty} 2 = 2$$

so $2n^2$ does not dominate n^2. But since the limit goes to a strictly positive constant, we can actually conclude that $f(n) = \Theta(g(n))$. To see this explicitly, consider again the defining inequality for Θ

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \iff c_1 \leq \frac{f(n)}{g(n)} \leq c_2$$

and take $c_\infty = \lim_{n\to\infty} \frac{f(n)}{g(n)}$ with $0 < c_\infty < \infty$. We can then always choose c_1 and c_2 such that

$$0 < c_1 < c_\infty < c_2 < \infty$$

and some n_0 so that the inequality if fulfilled for all $n > n_0$.

Example 4

Let $f(n) = \ln n$ and $g(n) = \sqrt{n}$. Here, we cannot easily tell whether the logarithm or the square root grows faster, i.e., what the value of the limit of $f(n)/g(n)$ would be. We therefore use L'Hôpital's rule, taking the derivative of $f(n)$ in the numerator and of $g(n)$ in the denominator and then consider the limit of the ratio of derivatives:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{\ln n}{\sqrt{n}} = \lim_{n\to\infty} \frac{\frac{1}{n}}{\frac{1}{2\sqrt{n}}} = \lim_{n\to\infty} \frac{2\sqrt{n}}{n} = \lim_{n\to\infty} \frac{2}{\sqrt{n}} = 0\ .$$

Thus, the square root dominates the logarithm and we have $\ln n = O(\sqrt{n})$.

Solving recurrence equations

See Compendium, part 5. We will only consider one approach to solving recurrence equations here.

Examples of recurrences with solutions

Example 1

$$T(n) = \begin{cases} 1 & \text{if} \quad n = 1 \\ T(n-1) + 1 & \text{if} \quad n > 1 \end{cases}$$

Solution: $T(n) = n$

Example 2

$$T(n) = \begin{cases} 1 & \text{if} \quad n = 1 \\ 2T(n/2) + n & \text{if} \quad n > 1 \end{cases}$$

Solution: $T(n) = n \lg n + n$

Proof by induction

Base case

$$n = 1 \implies n \lg n + n = 1 \lg 1 + 1 = 1 = T(1) \ .$$

Inductive step

1. We assume (*inductive hypothesis*) that $T(k) = k \lg k + k \ \forall k < n$.
2. We then have

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \qquad \text{by recurrence} && (4) \\ &= 2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) + n \qquad \text{by inductive hypothesis} && (5) \\ &= n \lg \frac{n}{2} + 2n && (6) \\ &= n(\lg n - \lg 2) + 2n \qquad \text{by properties of the logarithm} && (7) \\ &= n \lg n - n \lg 2 + 2n && (8) \\ &= n \lg n - n + 2n \qquad \text{because } \lg 2 = 1 \text{ by definition} && (9) \\ &= n \lg n + n \ . && (10) \end{aligned}$$

3. We have thus proven by induction that the solution holds for all n.

Solving recurrences using the "Master method"

General solution to recurrences of the form

$$T(n) = aT(n/b) + f(n) \qquad \text{where } a \geq 1, b > 1, f(n) > 0.$$

- Useful to solve many recurrences, but not all
- Note that a and b must be constants, *independent* of n
- $f(n)$ needs to fulfill certain requirements
- Solution has three cases, depending on how $f(n)$ relates to $n^{\log_b a}$

Case 1

Condition

$$f(n) = O(n^{\log_b a - \epsilon}) \quad \text{for some } \epsilon > 0$$

- $f(n)$ is polynomially smaller than $n^{\log_b a}$

Solution

$$T(n) = \Theta(n^{\log_b a})$$

- Runtime is dominated by leaves of recursion tree, internal operations contribute little

Case 2

Condition

$$f(n) = \Theta(n^{\log_b a})$$

- $f(n)$ grows as fast as $n^{\log_b a}$

Solution

$$T(n) = \Theta(n^{\log_b a} \lg n)$$

- Recursion tree has $\Theta(\lg n)$ levels, equal cost $\sim n$ at each level
- This case can be extended to $f(n) = \Theta(n^{\log_b a} \lg^k n)$, but that is not part of the curriculum for INF221 (for details, see, e.g., Wikipedia))

Case 3

Condition

$$f(n) = \Omega(n^{\log_b a + \epsilon}) \quad \text{for some } \epsilon > 0$$

and regularity condition

$$af(n/b) \leq cf(n) \quad \text{for some } c < 1 \text{ and all large } n$$

- $f(n)$ is polynomially greater than $n^{\log_b a}$
- Regularity condition always fulfilled if $f(n) = n^k$ is polynomial

Solution

$$T(n) = \Theta(f(n))$$

- Runtime dominated by root of recursion tree

Examples where master theorem can be applied

$$T(n) = aT(n/b) + f(n) \qquad \text{where } a \geq 1, b > 1, f(n) > 0.$$

Example 1

- $T(n) = 5T(n/2) + \Theta(n^2) \implies a = 5, b = 2$
- Compare $n^{\log_2 5}$ to n^2
- We have $\log_2 5 > 2 \implies \log_2 5 - \epsilon = 2 \implies$ Case 1
- $T(n) = \Theta(n^{\lg 5})$

Example 2

- $T(n) = 27T(n/3) + \Theta(n^3) \implies a = 27, b = 3$
- Compare $n^{\log_3 27} = n^3$ to n^3
- Case 2
- $T(n) = \Theta(n^3 \lg n)$

Example 3

- $T(n) = 5T(n/2) + \Theta(n^3) \implies a = 5, b = 2$
- Compare $n^{\log_2 5}$ to n^3
- We have $\log_2 5 < 3 \implies \log_2 5 + \epsilon = 3 \implies$ Case 3
- Regularity condition: $af(n/b) = 5(n/2)^3 = 5n^3/8 \leq cn^3$ for $c = 5/8 < 1$
- Case 3
- $T(n) = \Theta(n^3)$

Examples where master theorem cannot be applied

Example 1: non-constant a

- $T(n) = 2^n T(n/4) + \Theta(1) \implies a = 2^n, b = 4$
- a is not constant, so theorem does not apply

Example 2: regularity condition not fulfilled

- $T(n) = 2T(n/2) + n^2(2 - \cos n) \implies a = 2, b = 2, f(n) = n^2(2 - \cos n)$
- We have $n^{\log_b a} = n$ and $f(n) = n^2(2 - \cos n) \geq n^2$ because $-1 \leq \cos n \leq 1$
- Therefore, only case 3 may apply: we have $f(n) = \Omega(n^2)$, which fits the requirement for case 3 with $\epsilon = 1$
- Now consider the regularity condition:
  - We need to show that $af(n/b) \leq cf(n)$ for some fixed $c < 1$ and **all** $n > n_0$
  - We will show that we can for any n_0, always find some n that will break this condition
  - Choose some large n so that $n \approx 2(2k+1)\pi$
  - Thus $\cos n \approx 1 \implies f(n) \approx n^2$
  - We further have $af(n/b) = 2f((2k+1)\pi)$ and $\cos((2k+1)\pi) \approx -1$
  - Therefore $af(n/b) \approx 2(n/2)^2(2 - (-1)) = 6n^2/4 = \frac{3}{2}n^2$

    - The regularity condition then becomes $\frac{3}{2}n^2 \leq cn^2$
    - So for this n, we would need $c \geq \frac{3}{2}$, violating the requirement that $c < 1$. Since we clearly can find some $n \approx 2(2k+1)\pi$ everywhere along the number line, there is no n_0 beyond which the regularity condition would hold, and case 3 does not apply.
- Note that this is a pretty artifically constructed case.

Probabilistic analysis and randomized algorithms

See Compendium parts 6 and 7.

The technology problem

Note

The following example is losely based on the "hiring problem" example in CLRS. I consider the textbook example cynical and in conflict with the Nordic model of labor relations. The example here poses the same estimation problem in a more constructive context.

The problem

- You are running a software development company.
- At any time, you want to use the best available tool (Integrated development environment, IDE, e.g. PyCharm, Eclipse, ...) to achieve maximum productivity.
- Once you find that a new IDE is better than the one you are using now, you switch to the new IDE.

Algorithm

```
Select-IDE(n)
    current_ide = NIL
    ide_score   = -infinity
    for i = 1 to n
        new_score = evaluate(IDE i)
        if new_score > ide_score
            current_ide = IDE i
            ide_score = new_score
```

Cost model

- We test (over a given time) n IDEs
- We assume that in $m \leq n$ cases, the new IDE is better than the one we currently use
- What is the total cost for this strategy over time?
- Assumptions
  - Evaluating an IDE has cost c_E
  - Switching to a new IDE requires you to re-train your staff and incurs cost c_S
  - Switching will usually cost more than evaluating: $c_S > c_E$
- The cost is then given by

$$C(n, m) = nc_E + mc_S$$

Worst case

- Every IDE is better than the previous one
- We switch $m = n$ times
- Total cost $C(n, n) = nc_E + nc_S \approx nc_S$ ($c_E \ll c_S$)

Best case

- We chose the best IDE right at the start
- We switch only once
- Total cost $C(n, 1) = nc_E + c_S \approx nc_E$

Average case

- How do we handle the average case?
- What is the average case?
- We must make assumptions about how qualities of IDEs are distributed!
- We can then perform a *probabilistic analysis*
- In the problem here, n is fixed
- Only the number m of IDE switches and staff re-trainings varies

Indicator random variables

- Technique for computing the expected value of random variables

Definition

- Let S be a *sample space* and $A \in S$ an event
- Examples:
  - Single die: $S = \{1, 2, 3, 4, 5, 6\}$, A is result of single roll
  - Two dice: $S = \{2, \ldots, 12\}$, A is sum of two dice rolls
  - IDE evaluation: $S = \{\text{FALSE}, \text{TRUE}\}$, A is result of `new_score > ide_score`

- We define the *indicator random variable*

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{if } A \text{ does not occur} \end{cases}$$

Lemma

For an event A, let $X_A = I\{A\}$. Then $E[X_A] = \Pr\{A\}$.

Here

- $E[X_A]$ is the expected value of X_A
- $\Pr A$ is the probability of A

Proof

$$\begin{aligned} E[X_A] &= E[I\{A\}] && (11) \\ &= 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\bar{A}\} && (12) \\ &= \Pr\{A\} && (13) \end{aligned}$$

where $\bar{A} = S \setminus \{A\}$ is the complement of A, i.e., all events in S that are not A.

Simple example

Determine the expected number of heads when flipping a fair coin

- $S = \{H, T\}$
- $\Pr\{H\} = \Pr\{T\} = 1/2$
- Indicator variable $X_H = I\{H\}$
  - counts number of heads in one flip
- Using the lemma, we have $E[X_H] = \Pr\{H\} = 1/2$

More complicated example

Determine the expected number of heads in n coin flips

- Let X be the random variable for the number of heads in n flips.
- Define indicator random variables $X_i = I\{i^{\text{th}} \text{ flip results in event } H\}$ for $i = 1, \ldots, n$
- Then

$$X = \sum_{i=1}^{n} X_i$$

- and the expected number of heads is

$$E[X] = E\left[\sum_{i=1}^{n} X_i\right]$$

- Because expectation is *linear*, we have

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^{n} X_i\right] && (14) \\ &= \sum_{i=1}^{n} E[X_i] && (15) \\ &= \sum_{i=1}^{n} 1/2 && (16) \\ &= n/2\,. && (17) \end{aligned}$$

  where we used $E[X_i] = \Pr\{H\} = 1/2$ for $i = 1, \ldots, n$ from the lemma.

Application to technology example

- We assume that IDEs arrive in a random order, i.e., that the sequence of evaluation scores for the IDEs is random.
- Let X be the random variable that equals the number of times we switch IDEs.
- We define indicator variables for $i = 1, \ldots, n$

$$X_i = I\{\text{we switch to IDE i}\} = \begin{cases} 1 & \text{if we switch to IDE } i \\ 0 & \text{if we don't switch to IDE } i \end{cases}.$$

- We have
  - $X = X_1 + X_2 + \cdots + X_n$
  - $E[X_i] = \Pr\{\text{we switch to IDE } i\}$
- We need to obtain $\Pr\{\text{we switch to IDE } i\}$

Probability of switching to i^{th} candidate IDE

To find the probability of switching to the ith candidate IDE we consider, we proceed as follows:

- Let q_i be the quality (evaluation score) for IDE i and assume all IDEs have different scores.
- Assume that we test the IDEs in random order in the sense that the qualities q_i are not ordered in any way.
- We certainly select the first IDE we consider, so we have $\Pr\{\text{switch to } 1\} = 1$.
- We will select the second IDE only if it is better than the first, i.e., if $q_2 > q_1$.
  - There are exactly two possible outcomes for this test.
  - Because we assume random ordering of the IDE qualities, each outcome is equally likely.

    - Therefore, $\Pr\{\text{switch to } 2\} = 1/2$.
- For the third IDE, we will only switch to it if $q_3 > q_2 \wedge q_3 > q_1$, i.e., if q_3 is the largest number among $\{q_1, q_2, q_3\}$.
    - Since these numbers are in random order, the probability that the last out of three numbers is the largest one is $1/3$.
    - Therefore, $\Pr\{\text{switch to } 3\} = 1/3$.
- Continuing this train of thought, we will only switch to the ith if $q_i > q_j$ for all $j = 1, \ldots, i-1$.
    - Because we assume random ordering, every one of the i numbers $q_1, \ldots, q_i$ is the largest with equal probability.
    - That probability is $1/i$.
- Therefore

$$\Pr\{\text{we switch to IDE } i\} = \frac{1}{i}\ .$$

The probability to switch thus drops the more IDEs we have checked. This makes sense: The more IDEs we have seen, the more good ones we have seen (assuming random ordering) and therefore the likelihood of seeing even better ones diminishes.

Average cost of switching IDEs

Exploiting linearity, we have

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^{n} X_i\right] && (18)\\ &= \sum_{i=1}^{n} E[X_i] && (19)\\ &= \sum_{i=1}^{n} \frac{1}{i} && (20)\\ &= \ln n + O(1)\,. && (21) \end{aligned}$$

In the last step we exploited that

$$\sum_{i=1}^{n} \frac{1}{i} = H_n$$

is the nth Harmonic number and that

$$\lim_{n\to\infty} H_n = \ln n + \gamma$$

where $\gamma \approx 0.57722$ is the Euler–Mascheroni constant.

Thus, the number of IDE switches, $m = E[X]$, will in the average case grow logarithmically with n, the number of IDEs tested. We thus have for the cost

$$C_{\text{avg}}(n) = c_E n + c_S \ln n \approx c_S \ln n \ ,$$

where we assume in the last step that $c_E \ll c_S$ and n within reasonable limits. Thus, the average-case cost grows much more slowly than the worst-case cost nc_S.

Randomized algorithms

- For IDE-selection algorithm, random input provides much better performace than worst case ($\Theta(\ln n)$ vs $\Theta(n)$)
- We have shown that by analysing a *deterministic* algorithm using *probabilistic* methods
- Question: Could we avoid the worst case?
  - Worst case occurs when IDEs are evaluated in order from worst to best
  - We do not know beforehand how IDEs are ordered
  - But: if we randomize evaluation order, we will never be worse than $\Theta(\ln n)$
- Solution: *randomized algorithm*
  - Randomize evaluation order
  - Requires random permutation of $1, \ldots, n$

Randomly permuting an array in place

Algorithm

- Input: An array $A = \langle a_1, a_2, \ldots, a_n \rangle$ of numbers
- Output: Same array A, but with numbers a_i in random order

```
Randomize-in-place(A)
    n = A.length
    for i = 1 to n
        swap A[i] with A[Random(i, n)]
```

where `Random(i, n)` returns $k \in \{i, i+1, \ldots, n\}$ drawn with equal probability $p = 1/(n-i+1)$.

Run-time analysis

The algorithm requires

- n swaps
- n random numbers

Assuming that the run-time for `Random(i, n)` is constant, i.e., $\Theta(1)$, the run-time of the algorithm is thus $\Theta(n)$.

Proof of correctness

Permutations of a set

Let S be a finite set with n elements. Then any *ordered sequence* of all elements of S is a permutation of S.

In class exercise

Write down all permutations of the set $S = \{a, b, c\}$.

Number of permutations

A set S of $n = |S|$ elements has $n!$ permutations.

Proof

We build the permutation element by element from the set S. For the first element of the permutation we can choose between n elements, for the second between $n-1$ elements, and for the last element we have only 1 choice. Therefore, the total number of permutations is

$$n(n-1)(n-2)\cdots 2 \cdot 1 = n! \ .$$

k-permutation of a set

Let S be a finite set with n elements. Then any *ordered sequence of $k \leq n$ elements* selected from this set without replacement is a k-permutation of S. For given n and $k \leq n$, there are

$$\frac{n!}{(n-k)!}$$

k-permutations.

In class exercise

Write out all 2-permutations of the set $S = \{a, b, c\}$.

Proof of number of k-permutations in a set

We build the k-permutation element by element from set S. For the first element, we can choose between n elements, for the second between $n-1$, and so on, and for the kth element between $n-(k-1) = n-k+1$ elements. Therefore, we have

$$n(n-1)(n-2)\cdots(n-k+1) = \frac{n!}{(n-k)!}$$

possible k-permutations.

Aside: k-combinations of a set

Besides *k-permutations*, one often considers also *k-combinations* in combinatorics. A *k-combination* is any unordered subset of an n-element set S formed by drawing from S without replacement. There are

$$\frac{n!}{k!(n-k)!} = \binom{n}{k}$$

different k-combinations for an n-elements set.

Invariant

```
1: Randomize-in-place(A)
2:    n = A.length
3:    for i = 1 to n
4:        swap A[i] with A[Random(i, n)]
```

Immediately before the ith iteration of the loop on lines 3–4, the subarray $A[1..i-1]$ contains any of the possible $(i-1)$-permutations of the n elements in A with probability $(n-i+1)!/n!$.

Initialization

We have $i = 1$, i.e., we consider the empty array $A[1..0]$. This is the only zero-permutation of A, and it occurs with probability 1, consistent with $(n-1+1)!/n! = n!/n! = 1$. We have thus proven the initialization.

Maintenance

We assume that the invariant holds just before the ith iteration, i.e., $A[1..i-1]$ contains each of the possible $(i-1)$-permutations of A with equal probability $(n-i+1)!/n!$.

- With the next step, we generate a particular i-permutation $\langle x_1, x_2, \ldots, x_{i-1}, x_i \rangle$.
- Split this into $\langle x_1, x_2, \ldots, x_{i-1} \rangle$, and x_i
- Let E_1 be the event that the algorithm has placed our particular $(i-1)$-permutation $\langle x_1, x_2, \ldots, x_{i-1} \rangle$ in $A[1..i-1]$
- By induction hypothesis, $\Pr\{E_1\} = (n-i+1)!/n!$
- Let E_2 be the event that the algorithm places our particular x_i in $A[i]$
- The algorithm selects the new value in $A[i]$ at random from $A[i..n]$, whence $\Pr\{E_2\} = 1/(n-i+1)$
- Finding $\langle x_1, x_2, \ldots, x_{i-1}, x_i \rangle$ in $A[1..i]$ is equivalent to E_1 *and* E_2 happening together
- Thus, we need to obtain $\Pr\{E_1 \cap E_2\}$
- Using
  1. Bayes' theorem
  2. that we select $A[i]$ at random from $A[i..n]$, so that $\Pr\{E_2 | E_1\} = \Pr\{E_2\}$
  3. and the induction hypothesis we have

$$
\begin{aligned}
\Pr\{E_1 \cap E_2\} &= \Pr\{E_2|E_1\}\Pr\{E_1\} && (22)\\
&= \Pr\{E_2\}\Pr\{E_1\} && (23)\\
&= \frac{1}{n-i+1}\frac{(n-i+1)!}{n!} && (24)\\
&= \frac{(n-i)!}{n!}\ . && (25)
\end{aligned}
$$

This is exactly the probability required by the invariant at the beginning of the $(i+1)^{\text{st}}$ iteration, so we have proven the maintenance of the induction hypothesis.

Termination

We now have $i = n+1$, whence the invariant states that $A[1..n]$ contains any of the possible n-permutations with probability $(n-(n+1)-1)!/n! = 0!/n! = 1/n!$. Since an n-permutation is a permutation of the entire array, and each possible permutation occurs with the same probability $1/n!$, we have proven that the algorithm generates uniform random permutations of A.

Generating random numbers

"Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin." — John von Neumann (1951)

Literature

- Classiscal treatment: Donald Knuth, The Art of Computer Programming, vol. 2
- Textbook: JE Gentle, Random number generation and Monte Carlo methods
- Summary: HE Plesser, Generating Random Numbers, in S. Grün, S. Rotter, *Analysis of Parallel Spike Trains*, 2010

Types of random number generators

Physical random number generators

- The only source of true randomness
- E.g. radioactive decay, shot noise in electronic circuits
- Main problem: low rate of random numbers
- Not much used in practice

Pseudo-random number generators

- Based on some mathematical approach to generating sequences of "unordered" numbers

- *Pseudo*-random, not truly random, since sequences are generated by deterministic algorithms
- Typically generate integers uniformly distributed on $0, 1, \ldots M$
- Challenges in creating PRNGs:
  - fast
  - long period
  - uniformity of distribution
  - no correlations between numbers
- Long considered a "dark art", but much better theoretical understanding and control over the past 20 years
- Most important classes
  - Recurrence-based generators
  - Cryptographic generators

Quasi-random number generators

- Carefully crafted random numbers to sample a space
- Used much for numerical integration in high-dimensional spaces
- Also known as low-discrepancy series
- Example: Sobol sequence
- Not discussed further here

Recurrence-based generators

- Most common type of PRNG for scientific applications/simulations
- Based on repeated application of mathematical rule
- At any time, the generator is fully described by *state variable* Σ
- **Seed** (initialize):

$$\Sigma_0 = \mathcal{S}(S)$$

  - seed S
  - seeding function $\mathcal{S}$
- **Iterate**:

$$\Sigma_j = \mathcal{I}(\Sigma_{j-1})$$

  - iteration function $\mathcal{I}$
- **Draw**:

$$r_j = \mathcal{D}(\Sigma_j)$$

  - r_j is r-th random number generated
  - drawing function $\mathcal{D}$
- For simple generators, Σ is an integer and $\mathcal{S}$ and $\mathcal{D}$ are the identity function

Example: Linear Congruential Generator (LCG)

- **State variable**: Σ, typically 32 bit integer
- **Seed function**: $\mathcal{S}(s) = s$
- **Drawing function**: $\mathcal{D}(\Sigma) = \Sigma$
- **Iteration function**: $\mathcal{I}(\Sigma) = (a\Sigma + c) \bmod m$
- Choice of constant a, c, m is **essential**
- Popular choice (Lewis et al, 1969)
  - $a = 7^5 = 16807$
  - $c = 0$
  - $m = 2^{31} - 1$
  - Largest number that can be generated:
    $M = m - 1 = 2^{31} - 2 = 2,147,483,646$
- For these choices, the generator has maximal *period* $\rho = M$
  - all numbers will be drawn before any number repeats

Python implementation

In [1]:
```
import matplotlib.pyplot as plt
%matplotlib inline
```

In [2]:
```
plt.rcParams['figure.figsize'] = (15, 5)
plt.rcParams['text.usetex'] = True
```

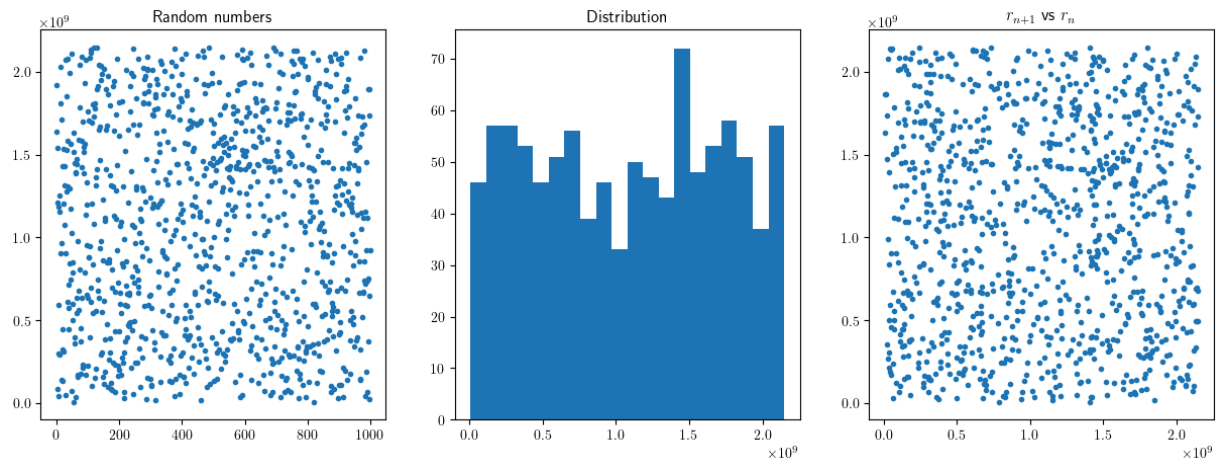
In [3]:
```
class LCG:
    a = 16807
    c = 0
    m = 2**31 - 1

    def __init__(self, seed):
        self.sigma = seed

    def draw(self):
        self.sigma = ( self.a * self.sigma + self.c ) % self.m
        return self.sigma
```

In [4]:
```
lcg = LCG(51223423)
r = [lcg.draw() for _ in range(1000)]
```

In [5]:
```
plt.subplot(1, 3, 1)
plt.plot(r, '.');
plt.title('Random numbers');
plt.subplot(1, 3, 2)
plt.hist(r, bins=20);
plt.title('Distribution');
plt.subplot(1, 3, 3)
plt.plot(r[:-1], r[1:], '.');
plt.title('$r_{n+1}$ vs $r_n$');
```



- Numbers seem unstructured and evenly distributed
- But: Short period
  - How quickly does a 3 GHz computer exhaust 2×10^9 random numbers?
  - Experience indicates that one should use only $\sqrt[3]{\rho} \approx 1290$ numbers to avoid correlations
- Also fails advanced tests for random number generators
- Note: Must be careful to avoid integer overflow when implementing $(a\Sigma + c) \bmod m$ in C, C++, Java, using 32 bit integers!

Multiple recursive generators

- Combine multiple recursion equations
- Recurrance based not only on previous state, but multiple earlier states
- Example: MRG32k3a (L'Ecuyer, 1999)

$$\Sigma_{1,n} = (1403580 \times \Sigma_{1,n-2} - 810728 \times \Sigma_{1,n-3}) \bmod m_1 \tag{26}$$
$$\Sigma_{2,n} = (527612 \times \Sigma_{2,n-1} - 1370589 \times \Sigma_{2,n-3}) \bmod m_2 \tag{27}$$
$$r_n = (\Sigma_{1,n} - \Sigma_{2,n}) \bmod m_1 \tag{28}$$
$$m_1 = 2^{32} - 209 = 4294967087 \tag{29}$$
$$m_2 = 2^{32} - 22853 = 4294944443 \tag{30}$$

- Two state variables Σ_1, Σ_2, each 53 bit
- Based on 3 previous Σ values
- Non-trivial draw function
- Period $\rho = (m_1^3 - 1)(m_2^3 - 1)/2 \approx 2^{191} \approx 10^{57}$
- Passes all test in TestU01 suite (2007)

Mersenne Twister

- Invented by Matsumoto and Nishimura (1998)
- So-called *feedback shift register* generator
- All arithmetic modulo 2, i.e., on bits

- State variable $\mathbf{\Sigma}$ is a vector of k 0s and 1s
- Iteration and drawing rules:

$$\mathbf{\Sigma}_j = \mathbf{A}\mathbf{\Sigma}_{j-1} \tag{31}$$
$$\mathbf{u}_j = \mathbf{B}\mathbf{\Sigma}_j \tag{32}$$
$$r_j = \sum_{l=1}^{w} u_{j,l} 2^{-l} \tag{33}$$

- Iteration matrix $\mathbf{A}$ is $k \times k$
- Output matrix $\mathbf{B}$ is $w \times k$, usually $w = 53 \ll k$
- Most-used variant has $k = 19937$
- Period $\rho = 2^{19937} - 1 \approx 10^{6000}$
- Passes almost all tests
- Widely used, e.g., in Python's `random` module

Permuted congruential generators

- Recent development (Melissa O'Neill, 2014)
- Solves some statistical problems in the Mersenne Twister and similar generators
- Much smaller state
- PCG64 generators is default in NumPy v1.17 and later

Cryptographic generators

- Problems with normal generators
  - Once state is know, we can predict all future random numbers
    - Extremely dangerous in any cryptographic conditions where unpredictability is essential
  - Can often not "skip ahead" in random number stream
    - Interesting in parallel simultions, where each of n processes wants to use only each n-th random number
- Cryptographic generators use standard encryption algorithms (e.g. AES)
- Algorithms use key K to encrypt plaintext P

$$C = E(K, P)$$

- We can generate random numbers by chosing seed S and drawing

$$C_j = E(K, S + j)$$

- Note that there is *no recurrence*: C_j does not depend on C_{j-1}

Advantages

- As long as key K is secret, we cannot predict future numbers from past numbers

- Skipping is trivial: we can obtain *any* number in the sequence at *any* time

Disadvantages

- Often slower than recurrent algorithms
  - But many newer CPUs provide hardware support for AES generators, making them fast
- Choice of K and S controls sequence of numbers
  - May require explicit rules for source of K and S to avoid manipulation
  - Applies also to recurvise algorithms (choice of seed)

A little more randomness: building a random number from bits

Given a random number generating function $\mathrm{Random}(0,1)$, which returns 0 and 1 with equal probability, provide an algorithm $\mathrm{Random}(0,b)$ that returns uniformly distributed random integers from $\{0,1,\ldots,b\}$, where $b=2^n-1$!

Correct solution building the random number as a binary number

```
Random(0, b)

r = 0
n = lg(b+1)
for k = 1..n
  r = r + RANDOM(0, 1) * 2^(k-1)
return r
```

Expected value of r

We first consider the problem mathematically. Each possible value of $r=\{0,1,2,\ldots b\}$ shall be selected with equal probability $p=1/(b+1)$. Therefore, the expected value of r is

$$E[r]=\sum_{r=0}^{b} rp=\frac{1}{b+1}\sum_{r=0}^{b} r=\frac{1}{b+1}\frac{b(b+1)}{2}=\frac{b}{2}\ . \tag{34}$$

Now we use indicator variables to check what the algorithm above does. We define

$$X_k=I\{\text{The } k\text{-th number returned by RANDOM(0,1) is } 1\}\ . \tag{35}$$

Then we have

$$r = \sum_{k=1}^{n} X_k 2^{k-1} \tag{36}$$

and thus

$$E[r] = E\left[\sum_{k=1}^{n} X_k 2^{k-1}\right] = \sum_{k=1}^{n} E[X_k] 2^{k-1} \tag{37}$$

$$= \frac{1}{2}\sum_{k=1}^{n} 2^{k-1} = \frac{1}{2}\sum_{k=0}^{n-1} 2^{k} \tag{38}$$

$$= \frac{2^n - 1}{2} = \frac{b}{2}\ , \tag{39}$$

where we have used the definition of b in the last step.

Proof of uniform distribution

The analysis above only proved the correctness of the mean of the numbers generated, but not their distribution. We will now see how to prove that all possible numbers are drawn with equal probability $p = 1/(b+1) = 1/2^n$.

Consider any integer m with $0 \leq m \leq b = 2^n - 1$, i.e., any number that may be returned by `RANDOM(0, b)`. This m has a *unique* binary representation

$$m = \sum_{k=1}^{n} m^{(k)} 2^{k-1}\ . \tag{40}$$

Using the X_k from above, we then have for the probablity to observe m

$$\Pr\{r = m\} = \Pr\{X_1 = m^{(1)} \wedge X_2 = m^{(2)} \wedge \cdots \wedge X_n = m^{(n)}\} \tag{41}$$

$$= \Pr\{X_1 = m^{(1)}\}\Pr\{X_2 = m^{(2)}\}\cdots\Pr\{X_n = m^{(n)}\} \tag{42}$$

$$= \prod_{k=1}^{n} \Pr\{X_k = m^{(k)}\} = \frac{1}{2^n} = \frac{1}{b+1}\ . \tag{43}$$

Here we used that the X_k are independent of each other so the joint probability can be expressed as product of the individual probabilities and that $\Pr\{X_k = m^{(k)}\} = 1/2$ by definition. Thus, we have proven that random numbers are equally distributed.

An incorrect solution

One could also consider the following algorithm

```
Random(0, b)
r = 0
for k = 1..b
```

```
    r = r + RANDOM(0, 1)
  return r
```

This algorithm clearly can generate random numbers from 0 to b and one can show that $E[r] = b/2$ (using indicator random variables). But the distribution is not uniform as can be shown as follows: For $b = 2$, we execute the loop twice and obtain

- $r = 0$ if and only if `RANDOM(0, 1)` returns 0 both times,
- $r = 2$ if and only if `RANDOM(0, 1)` returns 1 both times,
- $r = 1$ if and only if either the first or the second call to `RANDOM(0, 1)` returns 1.

Therefore, there is exactly one way each to obtain $r = 0$ and $r = 2$, but two ways to obtain $r = 1$, so $r = 1$ will occur with greater probability than the other two values, violating the requirement to have a uniform distribution.

In general, the algorithm returns the number of heads obtained when flipping a fair coin b times. This corresponds to a sequence of $n = b$ Bernoulli trials with probability of success $p = \frac{1}{2}$ and the number of successes k follows the binomial distribution

$$P(X = k) = \binom{n}{k} \times \frac{1}{2^n}$$